Ghost Resource Exterminator

Project Architecture & Design Documentation

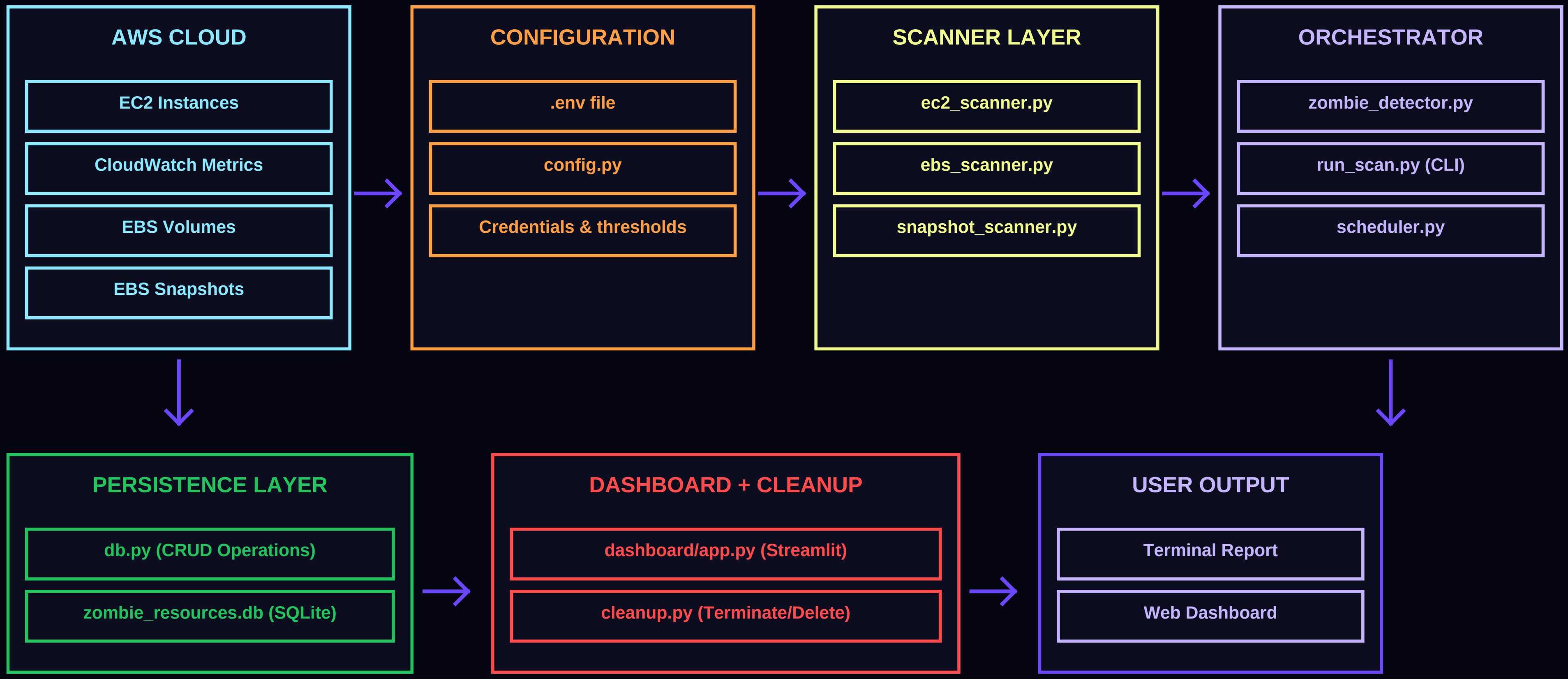
Data Flow | Module Diagram | ER Diagram | Table Design

Python 3.11+ | Boto3 / AWS | SQLite | Streamlit | CloudWatch

A Cloud FinOps tool that hunts down zombie AWS resources eating your budget

Data Flow Diagram

How data moves from AWS Cloud -> Scanners -> SQLite -> Dashboard -> User



Module Dependency Diagram

Import relationships between all Python modules (arrows show 'depends on')

Arrows indicate dependency direction: upper modules import/depend on lower modules

ER Diagram

Entity-Relationship: AWS resources (external) are scanned into zombie_resources table

AWS EC2 Instance

```
instance_id : TEXT (PK)
state : TEXT
cpu_utilization : REAL
```

External - not stored locally

AWS EBS Volume

```
volume_id : TEXT (PK)
state : TEXT
size_gb : INTEGER
volume_type : TEXT
```

External - not stored locally

AWS EBS Snapshot

```
snapshot_id : TEXT (PK)
start_time : DATETIME
volume_size : INTEGER
```

External - not stored locally

scanned as

scanned as

scanned as

zombie_resources (SQLite Table)

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique row identifier
resource_id	TEXT	NOT NULL	AWS resource ID (i-0abc, vol-0xyz)
resource_type	TEXT	NOT NULL	EC2 | EBS | Snapshot
region	TEXT	NOT NULL	AWS region name (e.g. us-east-1)
utilization	REAL	DEFAULT 0.0	CPU % for EC2; 0/100 for others
status	TEXT	NOT NULL DEFAULT 'Unknown'	Zombie | Active
reason	TEXT	(nullable)	Human-readable explanation
detected_at	TEXT	NOT NULL	ISO-8601 UTC timestamp

Key SQL Queries & Operations

All database operations used by the application through db.py

`save_resource()`

```
INSERT OR REPLACE INTO zombie_resources (...) VALUES (?,...)
```

Insert or update a resource record after scanning

`get_all_resources()`

```
SELECT * FROM zombie_resources ORDER BY detected_at DESC
```

Fetch all stored resources for the dashboard

`get_zombie_resources()`

```
SELECT * ... WHERE status = 'Zombie' ORDER BY ...
```

Return only zombie-classified resources

`get_summary_stats()`

```
SELECT resource_type, status, COUNT(*) GROUP BY ...
```

Aggregate counts for dashboard metric cards

`delete_resource(id)`

```
DELETE FROM zombie_resources WHERE resource_id = ?
```

Remove a resource after successful termination

`clear_resources()`

```
DELETE FROM zombie_resources
```

Wipe all records before a fresh scan

Zombie Detection Rules

How each resource type is classified as Zombie or Active

EC2 Instances

Detection Method

CloudWatch CPUUtilization averaged over 7 days

Zombie Threshold

Average CPU < 1%

Utilization Value

Actual CPU percentage (0.0 - 100.0)

Scanner File

scanner/ec2_scanner.py

Config Variable

EC2_CPU_ZOMBIE_THRESHOLD (default: 1.0)

EBS Volumes

Detection Method

Check Volume State via DescribeVolumes API

Zombie Threshold

State = 'available' (unattached)

Utilization Value

0.0 (zombie) / 100.0 (active)

Scanner File

scanner/ebs_scanner.py

Config Variable

N/A (built-in AWS state check)

EBS Snapshots

Detection Method

Check snapshot age from StartTime field

Zombie Threshold

Age > 30 days since creation

Utilization Value

0.0 (zombie) / 100.0 (active)

Scanner File

scanner/snapshot_scanner.py

Config Variable

SNAPSHOT_AGE_DAYS (default: 30)

All thresholds are configurable via .env variables (EC2_CPU_ZOMBIE_THRESHOLD, SNAPSHOT_AGE_DAYS, CLOUDWATCH_DAYS)

Estimated Monthly Cost Savings

Idle t3.medium EC2	**~$30/month**
100 GiB unattached gp3 EBS	**~$10/month**
1 TB old snapshots	**~$25/month**
10 zombie EC2 + 20 EBS + 50 snaps	**~$800-2000+/month**